

CS1010E: Programming Methodology

Assignment 1 (2 CA Marks)

Instructions

Submission Instructions

Read **ALL** instructions carefully.

1. Submission Content:

- Each question will be given a template file (*e.g.*, `aN_qM_template.py`).
 - You should work on your file locally before submitting to Coursemology.
- Comment out all the given **test cases** and **import** statements before copying the content of the entire file to Coursemology, *replacing* any content on Coursemology:
 - On template file: **CTRL + A** followed by **CTRL + C** (*i.e.*, *copy all*).
 - On Coursemology: **CTRL + A** followed by **CTRL + V** (*i.e.*, *replace all*).
- If any implementation is given, you are not allowed to change them.
- You are not allowed to *add*, *remove*, or *modify* any parameters from functions.
- You are not allowed to change the *name* of the given function.
- Unless otherwise stated, you are not allowed to add/change **import**.
- **You are not allowed to use syntax not covered in the module.**

2. Finalizing Submissions:

- You are responsible for finalizing your submission.
- Late submission will be deducted 33% for each 24 hour (*i.e.*, you get 0 mark if you are late for more than 48 hours).
- You should not submit at the last minute since Coursemology is not designed to handle large workload.
- If you see that Coursemology is *under maintenance*, it is due to large workload and not an actual maintenance. No extension will be given.

3. Testing:

- Do not call the required functions when you copy the code to Coursemology as it will be called automatically during testing.
- Your function should **return** the result instead of **print**.
- Test cases are called in succession (*i.e.*, *no reverting of global state*).
- Passing all test cases (*public or private*) does not guarantee full marks as we will do manual evaluation on other aspects such as code quality.
- You may assume that the code will only be tested on valid inputs that follows assumption. Do NOT put any checks for inputs with respect to the assumptions.

4. Good Programming Practice:

- Codes that are redundant or too long may be penalized.
- Restrictions must not be violated.
- Each test case (*public or private*) should complete within a reasonable amount of time typically within 1s to 2s.

Failure to follow any of the instructions above will result in deduction of your marks.

Files

- Assignment01.pdf
- a1_q1_template.py
- ⋮
- a1_q11_template.py

Coursemology

- Assignment 1 (2 CA Marks)

There are two compulsory questions in this assignment, Q1 and Q6. Each question is worth 1 CA mark.

The rest of the questions in the assignment are for your practice. You are encouraged to attempt as many question as possible, to improve your programming skills.

Questions

1. Body Mass Index [1 CA mark]
2. Type of Triangle
3. Factorial
4. Digit Sum
5. Perfect Number
6. Who are the Winners? [1 CA marks]
7. Doubling Every Character
8. Drawing Regular Polygon
9. Drawing Flower Shape

Question 1: Body Mass Index

[1 CA mark]

Given a person's weight (in kilograms) and height (in meter), his/her BMI (Body Mass Index) is calculated based on this formula:

$$BMI = \frac{Weight}{Height^2}$$

Write a function `compute_bmi()` that reads in that order, the user's **height** (in meter) and **weight** (in kilograms), and returns a string that shows the body type of the user:

1. **Under** : if the *BMI* is lower than 18.5 (*exclusive*)
2. **Normal** : if the *BMI* is higher than 18.5 (*inclusive*) but lower than 25 (*exclusive*)
3. **Over** : if the *BMI* is higher than 25 (*inclusive*) but lower than 35 (*exclusive*)
4. **Obese** : if the *BMI* is higher than 35 (*inclusive*)

For example, suppose height is 1.7 (meters) and weight is 68 (kilograms), function call `compute_bmi()` will read 1.7 and 68 from keyboard and then return the string **Normal**.

Be reminded that both **height** and **weight** should be converted to `float` type.

Note that in this question, we read data from keyboard using `input()`. For other questions in this assignment, data are passed to our functions as parameters (i.e. they don't use `input()`). These are two different designs.

Sample Run #1

```
1 >>> print(compute_bmi())
2 Enter height: 1.7
3 Enter weight: 68
4 Normal
```

Question 2: Type of Triangle

Write a function `triangle` that accepts three positive integers as inputs, denoting lengths of three sides of a triangle. The function will return the triangle's type:

1. `Not a triangle` : the length of any side is longer or equal to the sum of the other two sides.
2. `Equilateral` : all sides of a triangle have the same length.
3. `Isosceles` : two sides are equal in length and not equilateral.
4. `Scalene` : it is a triangle AND all sides are unequal (which also implies not isosceles and not equilateral).

For example, function call `triangle(5, 3, 3)` should return string `Isosceles`.

Question 3: Factorial

Write a function `factorial(n)` that takes an argument `n` and returns the factorial of `n` (i.e. $n!$).

Note that $n! = n * (n-1) * (n-2) * \dots * 1$.

For example, function call `factorial(5)` should return `120` ($= 5 * 4 * 3 * 2 * 1$).

Question 4: Digit Sum

Write a function `is_sum_odd(number)` that takes a non-negative integer `number` as parameter and returns `True` if the sum of digits of `number` is odd, or `False` otherwise.

For example, function call `is_sum_odd(12)` will return `True`, function call `is_sum_odd(110)` will return `False`, and function call `is_sum_odd(5)` will return `True`.

Note: use a loop to process the `number` digit by digit. Do not turn the `number` into a string.

Question 5: Perfect Number

A perfect number is a positive integer that is equal to the sum of its proper divisors. A proper divisor is a positive integer other than the number itself that divides the number evenly (i.e. no remainder).

For example, 6 is the smallest perfect number, because the sum of its proper divisors 1, 2, and 3 is equal to 6. 8 is not a perfect number because $1 + 2 + 4$ is not equal to 8.

Write a function `perfect_number(n)` that accepts a positive integer `n` and returns `True` if `n` is a perfect number, or `False` otherwise.

Question 6: Who are the Winners?

[1 CA marks]

Citizens of Zakadaha hold an annual Gagalafa festival to celebrate the harvest of their prized produce, the well-sought after cocoa beans Kokomoko. A lucky draw is held during the festival. Every participant is given a lucky draw number. Each year, the organizer decides on two non-zero digits:

1. The *factor-digit*

2. The *must-have-digit*

These two digits need not be distinct. A winning lucky draw number is a number that is a multiple of the *factor-digit* as well as contains the *must-have-digit*. Note that there could be more than one lucky draw winner.

Write a function `find_winners(factor, must_have, n)` that accepts the following three parameters:

1. The factor-digit (`factor`), which is a non-zero digit (*i.e.*, 1-9).
2. The must-have-digit (`must_have`), which is also a non-zero digit (*i.e.*, 1-9).
3. The number of participants (`n`) where the lucky draw numbers will be numbered from 1 to `n` (*inclusive*).

Your function should return the number of winners (*i.e.*, the number of participants whose lucky draw number is a multiple of *factor-digit* and contain the *must-have-digit*).

For example, if *factor-digit* is 3, *must-have-digit* is 5, and the number of participants is 100, then the number of winners is 6 (the winning numbers are 15, 45, 51, 54, 57 and 75).

You are advised to write a helper function `contain_digit(number, digit)` that returns `True` if number contains digit, or `False` otherwise. For example, function call `contain_digit(15, 5)` returns `True`. This function will be called by the `find_winners(factor, must_have, n)` function to check the existence of *must-have-digit*. Consult your tutor or lecturer, should you have any doubt on the helper function.

Restrictions

- You are **not** allowed to use any string functions or convert a number to string. However, you may write additional functions as you deem necessary.

Sample Run #1

```
1 >>> print(find_winners(3, 5, 100))
2 6
```

Sample Run #2

```
1 >>> print(find_winners(9, 1, 15))
2 0
```

Sample Run #3

```
1 >>> print(find_winners(7, 7, 200))
2 5
```

Question 7: Doubling Every Character

Write a function `double_char(word)` that accepts a String `word` as parameter and returns a new string in which every character of `word` is 'doubled'. For example,

- Function call `double_char('The')` returns `TThhee`
- Function call `double_char('AAbb')` returns `AAAAbbbb`

- Function call `double_char('Hi-There')` returns `HHii--TTthheerree`

Hint: the same problem solving strategy is used in Week 03 Extra Practice Q7.

Question 8: Drawing Regular Polygon Shape with Turtle

You have used the Python Turtle package to draw simple graphics. Some of the basic Turtle functions that you have come across includes:

1. `pd()` : *Pen down*. Any movements thereafter will be drawn.
2. `pu()` : *Pen up*. Any movements thereafter will not be drawn.
3. `forward(dist)` : Moves the turtle *forward* a number of units specified by the `dist`.
4. `backward(dist)` : Moves the turtle *backward* a number of units specified by the `dist`.
5. `right(deg)` : Turns/rotate the turtle *right/clockwise* by the specified `deg`.
6. `left(deg)` : Turns/rotate the turtle *left/anti-clockwise* by the specified `deg`.

Write the function `draw_reg_poly(d,n)` to draw a regular polygon with `n` sides where the length of each side is `d`. Note that a regular polygon is a polygon such that all angles are equal as well as all sides have the same length. An n -polygon has n number of sides. The initial position of the Turtle is shown in the two images below:

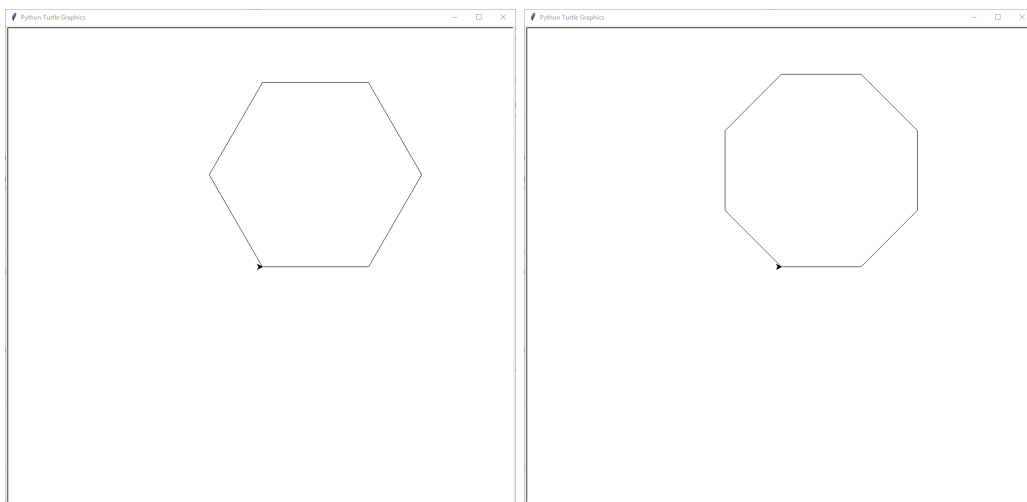


Figure 1: Sample run of `draw_reg_poly(200,6)` (left) and `draw_reg_poly(150,8)` (right).

Assumptions

- $d > 0$
- $n > 2$

Question 9: Flower

You will use the Turtle package to now draw a flower, which consists of petals arranged in a regular circular pattern such that the angle between each petal is the same. Each petal consists of a regular polygon.

Write the function `draw_flower(d,n,p)` to draw a flower pattern where `d` is the length of the regular polygon of `n` sides and `p` is the number of petals.

Figure 2 shows the drawing for function call `draw_flower(100, 8, 10)`. The shape consists of 10 petals where each petal is a regular octagon with 100-unit long sides (so each petal can be drawn using the `draw_reg_poly()` function from the previous question). Since there are 10 petals in the figure, each petal is shifted from another by 36° .

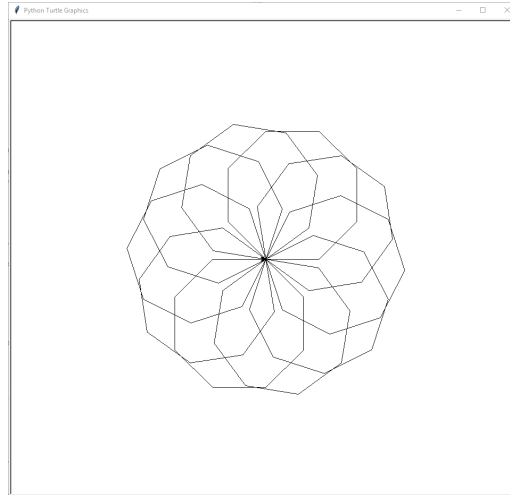


Figure 2: Sample run of `draw_flower(100, 8, 10)`.

Question 10: Invert Number

Write a function `invert_number(n)` that accepts a positive integer `n` as input parameter, reverses the order of each digit of `n` and returns the inverted value as an integer.

For example, if input integer is `12345`, the return value should be integer `54321`. As another example, if input integer is `12300`, the return value should be integer `321`.

Restrictions

- You are not allowed to use any string functions in this task.

Question 11: Reverse Numbers

Write a function `reversed_number(low, high)` that accepts two positive integers `low` and `high`, and returns the number of integers in the range `[low, high]`, both ends inclusive, whose reverse is the same as the original value. You may assume that `low < high`. For example, if `low` is 150 and `high` is 202, then there are 6 integers in the range `[150, 202]` whose reverse is the same as itself. They are: 151, 161, 171, 181, 191, and 202.

You are given the `invert_number(n)` function from the previous task (hidden from your view but will be executed by Coursemology together with your `reversed_number(low, high)` function).

Restrictions

- You are not allowed to use any string functions in this task.